

M17 / STUDY COMPANION

M17 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

DE17-01 | From requirements to architecture 3

DE17-02 | Batch vs streaming decisions 3

DE17-03 | Warehouse vs lakehouse 3

DE17-04 | Managed service vs build yourself 3

DE17-05 | Scaling and bottleneck reasoning 3

DE17-06 | Reliability and failure domains 4

DE17-07 | Cost-aware architecture 4

DE17-08 | Security and governance by design 4

DE17-09 | Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes 4

DE17-10 | Architecture review and defense 4

Read this first

Ready-made recall notes. Reconstruct the decision; do not rewrite the learner book.

DE17-01 | From requirements to architecture

Recall: Architecture starts before a service name. A good design turns business outcomes and non-functional requirements into explicit data contracts, latency targets, security boundaries, recovery goals and cost constraints. If you jump straight from 'we need a dashboard' to a list of products, the diagram may look professional while answering the wrong problem.

Ask yourself: Explain why architecture is primarily a requirements-to-trade-offs exercise rather than a cloud-service memorization exercise.

DE17-02 | Batch vs streaming decisions

Recall: Streaming is valuable when action loses value quickly, but it adds continuous state, ordering, replay, capacity and operational concerns. Batch is not 'old technology'; it is often the safest choice when latency requirements permit it. The architecture skill is choosing the simplest model that meets the decision deadline.

Ask yourself: Give one case where a streaming source still feeds a batch serving layer and explain why that is not contradictory.

DE17-03 | Warehouse vs lakehouse

Recall: Warehouse and lakehouse are not team identities; they are serving/storage patterns with different strengths. A warehouse often gives strong governed SQL analytics and operational simplicity. A lakehouse brings open-file/object-storage economics with table semantics and multi-engine flexibility. Many production platforms use both patterns or products that blur the boundary.

Ask yourself: Why is 'warehouse versus lakehouse' often a boundary-placement decision rather than a winner-takes-all decision?

DE17-04 | Managed service vs build yourself

Recall: Every custom component creates ownership: upgrades, security patches, scaling, backups, observability and specialist knowledge. Managed services trade some flexibility/control for reduced undifferentiated operations. The architecture question is where custom control creates enough business value to justify that ownership.

Ask yourself: What responsibilities disappear with a managed service, and which responsibilities always remain yours?

DE17-05 | Scaling and bottleneck reasoning

Recall: Scaling is not adding machines everywhere. Throughput is constrained by the narrowest effective boundary: source API quotas, network, partitions, shuffle, storage write rate, database concurrency, downstream service or even one serialized key. Scaling the wrong layer increases cost without increasing useful throughput.

Ask yourself: Why does scaling a non-bottleneck often produce almost no improvement?

DE17-06 | Reliability and failure domains

Recall: Reliable architecture assumes components fail. The design limits how far a failure spreads, preserves durable evidence, and defines recovery that can be tested. Availability is an end-to-end property: redundant compute does not help if every worker depends on one unrecoverable state store or one credential.

Ask yourself: Explain the difference between replication, backup and replay. Why do you sometimes need all three?

DE17-07 | Cost-aware architecture

Recall: Cost is an architecture constraint, not a finance cleanup step. Data systems spend across storage, compute, data movement, managed-service premiums, observability, redundancy and people. Optimization means paying for requirements intentionally, not simply choosing the lowest unit price.

Ask yourself: Why is cost optimization a trade-off discipline rather than simply reducing resource count?

DE17-08 | Security and governance by design

Recall: Security added at the end becomes a patchwork of broad permissions and manual exceptions. Architecture should decide identity boundaries, least privilege, encryption, data classification, network paths, auditability, retention and ownership before data is widely copied.

Ask yourself: What is the difference between authentication, authorization and governance? Give one architecture control for each.

DE17-09 | Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes

Recall: Job descriptions often mention CDC, data contracts, secrets platforms, data mesh and Kubernetes together. A junior engineer does not need five specialist certifications before applying, but should understand what problem each solves, where it fits, and what trade-offs or ownership it introduces. This lesson is a promotion map, not a replacement for dedicated platform engineering courses.

Ask yourself: In one sentence each, distinguish CDC, a data contract, Vault, data mesh and Kubernetes. Then name one dangerous category mistake.

DE17-10 | Architecture review and defense

Recall: Architecture interviews and real design reviews are not drawing competitions. Reviewers want to see whether you can explain requirements, boundaries, failure behavior, security, cost, alternatives and evidence. A defensible architecture makes assumptions visible and survives changed conditions without collapsing into 'because this tool is popular'.

Ask yourself: Give your architecture defense in this order: requirement, decision, evidence, trade-off, failure behavior. Why is that stronger than describing components left-to-right?